

06. 기술 스택 (Tech Stack & Architecture)

06. 기술 스택 (Tech Stack & Architecture)

문서 정보

버전: v1.0 | 작성일: 2026-05-03 | 상태: 초안 | 작성자: 경북대학교 AI/빅데이터 전문가 양성 과정 — TBD팀

한 줄 요약

Flutter (모바일) + FastAPI (백엔드) + Google Cloud Vision (OCR) + Claude API (LLM) + PostgreSQL/TimescaleDB (DB) 의 5개 핵심 스택을 중심으로, 학생 팀이 10주 안에 양대 스토어 배포 가능한 의료 헬스케어 앱을 구현할 수 있도록 설계된 하이브리드 아키텍처.

목차

- 1. 전체 아키텍처
- 2. 기술 스택 일람
- 3. 영역별 상세 — 선택 근거와 대안
- 4. 데이터 흐름 — 영양제 사진에서 결과까지
- 5. 비용 추정 (학생 팀 기준)
- 6. 의사결정 매트릭스 — 왜 이 스택인가
- 7. 학습 자료
- 8. 확장·마이그레이션 가능성

1. 전체 아키텍처

1.1 시스템 다이어그램



1.2 핵심 설계 원칙

원칙	의미
클라우드 우선	OCR·LLM은 자체 호스팅하지 않고 API 활용 → 학생 팀 부담 ↓
온디바이스 헬스 데이터	걸음수·심박수는 HealthKit/Health Connect로 직접 수집 → 백엔드 부하 ↓
백엔드 중심 비즈니스 로직	알고리즘은 모두 Python → 단위 테스트·디버깅 용이
API 키 보호	모바일에서 외부 API 직접 호출 금지 → 모든 외부 호출은 백엔드 경유
수평 확장 가능 구조	Stateless FastAPI + DB/Redis 분리 → 사용자 증가 시 수평 확장

2. 기술 스택 일람

2.1 핵심 5개 스택

영역	기술	버전	용도
모바일	Flutter	3.24+ (stable)	iOS + Android 동시 배포
백엔드	FastAPI (Python 3.11+)	0.110+	REST API 서버
OCR	Google Cloud Vision API	v1	영양제 라벨 텍스트 추출
LLM	Claude API (Anthropic)	claude-sonnet-4-6	텍스트 → 영양 성분 JSON 구조화
DB	PostgreSQL + TimescaleDB	PG 16 / TS 2.x	관계형 + 시계열 통합

2.2 부가 스택

영역	기술	용도
캐싱	Redis 7+	OCR 결과·KDRIs 룩업·세션 캐싱
헬스 데이터	health 패키지 (Flutter)	HealthKit + Health Connect 통합
컨테이너	Docker + Docker Compose	로컬 개발·배포 환경 통일
인프라	NCP / AWS / GCP (택1)	백엔드 호스팅
이미지 저장	NCP Object Storage / AWS S3	영양제·식단 사진 저장
모니터링	Python logging + Sentry (선택)	에러 추적
CI/CD	GitHub Actions	자동 테스트·빌드
API 테스트	Pytest + httpx	백엔드 통합 테스트
API 문서	FastAPI 자동 Swagger UI	<code>/docs</code> 엔드포인트

2.3 백엔드 주요 라이브러리 (requirements.txt 골격)

```
fastapi>=0.110
uvicorn[standard]>=0.27
pydantic>=2.6
sqlalchemy>=2.0
asyncpg>=0.29
alembic>=1.13      # DB 마이그레이션
redis>=5.0
httpx>=0.27        # 외부 API 호출
google-cloud-vision>=3.7
anthropic>=0.25    # Claude API
pillow>=10.2       # 이미지 처리
python-multipart>=0.0.9 # 파일 업로드
pytest>=8.0
pytest-asyncio>=0.23
pytest-cov>=4.1
black>=24.4
ruff>=0.4
mypy>=1.10
```

2.4 모바일 주요 패키지 (pubspec.yaml 골격)

```
dependencies:
  flutter:
    sdk: flutter

  # 상태 관리
  flutter_riverpod: ^2.5.1 # 또는 provider/bloc

  # API 통신
  dio: ^5.4.0
  retrofit: ^4.1.0

  # 헬스 데이터
  health: ^10.2.0 # HealthKit + Health Connect
  permission_handler: ^11.3.0

  # 이미지 처리
  image_picker: ^1.0.7 # 카메라, 갤러리
  image_cropper: ^5.0.1

  # UI
  flutter_screenutil: ^5.9.0 # 반응형
  fl_chart: ^0.66.0 # 차트

  # 라우팅
  go_router: ^13.2.0

  # 로컬 저장
  shared_preferences: ^2.2.2
  flutter_secure_storage: ^9.0.0 # 토큰 등 민감정보

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0
  build_runner: ^2.4.0
```

3. 영역별 상세 — 선택 근거와 대안

각 기술별로 왜 선택했는가(Why), 대안은 무엇인가(Alternatives), 트레이드오프(Tradeoffs)를 정리한다.

3.1 모바일 — Flutter

선택 근거

1. 단일 코드베이스로 iOS + Android 동시 배포 — 학생 팀의 인력 부담 ↓
2. Dart 학습 곡선이 완만 — Python 경험자에게 친숙한 문법
3. **health** 패키지 검증됨 — HealthKit + Health Connect를 동시 래핑
4. Hot Reload — 빠른 개발 사이클

5. 풍부한 UI 라이브러리 — Material + Cupertino 동시 지원

대안 비교

옵션	장점	단점	결론
Flutter ⭐	단일 코드 양대 OS, Hot Reload, Dart 친숙	네이티브 대비 약간 무거움	✅ 채택
React Native	JS 생태계, 인기 ↑	헬스 패키지 두 개 필요 (RN-Health + RN-Health-Connect)	❌
Native (Swift + Kotlin)	최고 성능	두 개 스택 학습·유지 부담	❌
Xcode 단독	iOS 네이티브	Android 배포 불가	❌

트레이드오프

- ⚠️ 앱 크기 커짐 (Flutter 엔진 ~10MB)
- ⚠️ 일부 네이티브 SDK는 채널 통신 코드 필요

3.2 백엔드 — Python 3.11+ + FastAPI

선택 근거

- 팀이 Python에 익숙 — 학습 비용 0
- FastAPI = Type Hint 기반 자동 문서화 — Pydantic 스키마 → Swagger UI 자동 생성
- 비동기 (async/await) — OCR/LLM API 호출은 I/O 바운드라 비동기가 큰 이점
- AI/ML 라이브러리 풍부 — pandas, numpy, scikit-learn 등 즉시 활용
- 알고리즘 검증 용이 — pytest + 단위 테스트 자연스러움

대안 비교

옵션	장점	단점	결론
FastAPI ⭐	비동기, 자동 문서, 타입 안전	신규 프레임워크 (3년+)	✅ 채택
Django REST Framework	성숙, 어드민 자동	무거움, 비동기 약함	❌
Flask	가벼움, 자유도 ↑	표준 부재, 타입 검증 수동	❌
Node.js (Express/NestJS)	JS 단일 언어	팀 학습 비용	❌
Spring Boot (Java)	엔터프라이즈 표준	학습 곡선 ↑↑	❌

트레이드오프

- ⚠️ Python GIL → CPU 바운드 작업은 Worker 프로세스 분리 필요 (uvicorn workers)
- ⚠️ 타입 시스템이 런타임 검증 — Pydantic으로 보완

3.3 데이터베이스 — PostgreSQL 16 + TimescaleDB

선택 근거

1. PostgreSQL = 사실상 표준 — JSON 지원, 확장성, 무료
2. TimescaleDB 확장으로 시계열 데이터(걸음수·체중) 효율 처리
3. JSON/JSONB 컬럼 — 영양제 성분 등 동적 스키마 저장
4. GIN 인덱스 — 식품 검색 성능
5. AES-256 컬럼 암호화 가능 — 의료 데이터 보안

데이터 모델 개요

관계형 (PG):

users (사용자 정보)
 supplements (영양제 마스터)
 foods (식품 마스터, KDRIs/식약처)
 user_supplements (사용자가 등록한 영양제)
 meals (식단 기록)
 diagnoses (만성질환 정보)

시계열 (TimescaleDB Hypertable):

step_counts (걸음수, hour 단위 집계)
 weight_logs (체중 측정, 일 단위)
 heart_rate_samples (심박수, 분 단위)

대안 비교

옵션	장점	단점	결론
PostgreSQL + TimescaleDB 🌟	표준 + 시계열 통합	시계열만 본다면 over-engineering	✅ 채택
MongoDB	동적 스키마	트랜잭션 약함, 의료 데이터 엔 부족	❌
MySQL	친숙	JSON 지원 약함, 확장성 ↓	❌
Firebase Firestore	모바일 통합 ↑	비용 폭증 우려, 복잡 쿼리 어려움	❌

트레이드오프

- ⚠️ 운영 부담 (인덱스, VACUUM, 백업) — 학생 단계에서는 매니지드 서비스 활용 권장 (NCP DB Manager / AWS RDS)

3.4 캐싱 — Redis 7+

선택 근거

1. OCR 결과 캐싱 — 같은 영양제 라벨 재인식 비용 ↓
2. KDRIs 록업 캐싱 — 자주 조회되는 영양 기준값
3. 세션·토큰 저장
4. Rate Limiting — Cloud Vision API 비용 폭주 방지

트레이드오프

- ⚠ 메모리 제한 — TTL 정책 필수 (예: OCR 결과 30일 캐싱 후 만료)

3.5 OCR — Google Cloud Vision API

선택 근거

1. 정확도 검증됨 — 한국어 + 영어 영양제 라벨에서 92~98% 보고
2. 첫 1,000건/월 무료 — PoC·MVP 단계에서 비용 0
3. 이후 \$1.50/1,000건 — 학생 팀 예산으로 충분
4. TEXT_DETECTION + DOCUMENT_TEXT_DETECTION 두 모드 — 영양제 라벨엔 후자 적합
5. Python SDK 성숙

대안 비교

옵션	정확도	한국어	비용	결론
Google Cloud Vision ★	92~98%	✅	\$1.5/1k (1k 무료)	✅ 채택
Naver CLOVA OCR	한국어 SOTA	★★	일정 무료 + 종량	🔄 백업 옵션
AWS Textract	95%+	△	\$1.5/1k	❌ (한국어 약점)
Azure AI Vision	90%+	✅	\$1/1k	△
Tesseract (오픈소스)	70~85%	△ (학습 필요)	무료	❌ (정확도 ↓)
PaddleOCR	85%+	✅	무료 (자체 호스팅)	△ (인프라 부담)

권고

- 주력: Google Cloud Vision API (DOCUMENT_TEXT_DETECTION 모드)
- 백업: Naver CLOVA OCR (Cloud Vision 정확도 부족 시 폴백)
- 자체 호스팅 옵션: PaddleOCR (장기적 비용 절감 시)

3.6 LLM — Claude API (Anthropic)

선택 근거

1. OCR 텍스트 → 구조화 JSON 변환에 강점 — 라벨 비정형 텍스트를 깔끔한 JSON 스키마로
2. 한국어 처리 능력 우수
3. 긴 컨텍스트 — 200K 토큰까지 (영양제 라벨 + 한국어 영양소 매핑 동시 처리 가능)
4. Tool Use (Function Calling) — Pydantic 스키마와 통합 용이
5. JSON Mode — 응답 포맷 강제 가능

사용 예시 (의사 코드)

```
import anthropic

client = anthropic.Anthropic()

# OCR 결과 텍스트
ocr_text = """
Supplement Facts
Vitamin C 1000 mg 1111% DV
Vitamin D3 25 mcg (1000 IU) 125% DV
...
"""

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{
        "role": "user",
        "content": f"""Extract supplement facts from the following OCR text.
Return JSON with schema:
{{
  "supplements": [
    {{
      "name": str, "amount": float, "unit": str, "daily_value_pct": float
    }}
  ]
}}

OCR text:
{ocr_text}
"""
    }],
    stop_sequences=["\n\n"]
)
```

대안 비교

옵션	한국어	가격	JSON 응답	결론
Claude (Anthropic) ★	우수	적정	✅ Tool Use	✅ 채택
GPT (OpenAI)	우수	비슷	✅ Function Calling	🔄 백업
Gemini (Google)	우수	저렴	✅ JSON Mode	△
Qwen / LLaMA (오픈소스)	△	무료 (자체 호스팅)	△	❌ (인프라 부담)

권고

- **주력:** Claude (이번 프로젝트 작성에도 사용된 모델 — 한국어·구조화 모두 검증)
- **백업:** GPT (Claude API 다운 시 자동 폴백)

3.7 헬스 데이터 — health 패키지 (Flutter)

선택 근거

1. HealthKit + Health Connect 동시 래핑 — 단일 코드로 양대 OS 지원
2. 걸음수 / 심박수 / 활동에너지 / 운동 / 체중 / 키 등 50+ 데이터 타입
3. 백그라운드 동기화 가능 (HKObserverQuery 등)
4. 권한 관리 통합 API

주의사항

- ⚠️ Health Connect는 사용자가 별도 앱 설치 필요 (Android 13 이하)
- ⚠️ Galaxy Watch는 Samsung Health → Health Connect 경유로 데이터 흐름
- ⚠️ 백그라운드 권한은 Apple/Google 심사 까다로움

3.8 인프라 — Docker + 클라우드 (NCP/AWS/GCP)

권장 구성

환경	도구
로컬 개발	Docker Compose (FastAPI + PostgreSQL + Redis)
CI/CD	GitHub Actions
이미지 레지스트리	Docker Hub (무료 1개 private) 또는 GitHub Container Registry
클라우드	NCP / AWS / GCP 중 택 1 (학생 크레딧 활용)

클라우드 옵션 비교

옵션	학생 혜택	한국 위치	추천 시나리오
NCP (Naver Cloud)	학생 크레딧 (정부 사업)	✅ 한국	🌟 한국 사용자 + 의료 데이터 보호
AWS	AWS Educate (제한적)	△ Tokyo / Seoul	글로벌 확장 시
GCP	\$300 무료 크레딧	△ Seoul	Google Cloud Vision 통합 시
Azure	Azure for Students	△	Microsoft 생태계 시

Docker Compose 골격 (`docker-compose.yml`)

```
version: "3.9"
services:
  backend:
    build: ./backend
    ports: ["8000:8000"]
    env_file: ./backend/.env
    depends_on: [postgres, redis]

  postgres:
    image: timescale/timescaledb:latest-pg16
    volumes: [pgdata:/var/lib/postgresql/data]
    environment:
      POSTGRES_DB: lemon
      POSTGRES_USER: lemon
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports: ["5432:5432"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

volumes:
  pgdata:
```

3.9 모니터링 — 로깅 + Sentry (선택)

도구	용도	무료 티어
Python logging	애플리케이션 로그	무한
Sentry (선택)	에러 자동 추적·알림	5,000 events/month 무료
Uptime Robot (선택)	서비스 가용성 모니터링	50개 모니터 무료

4. 데이터 흐름 — 영양제 사진에서 결과까지

4.1 시퀀스 다이어그램



4.2 단계별 처리

단계	작업	예상 소요 시간
1. 사용자 촬영	Flutter <code>image_picker</code>	—
2. 클라이언트 검증	이미지 크기·형식 검증 (5MB 이하 JPEG)	< 100ms
3. 백엔드 업로드	HTTPS multipart/form-data	200~500ms
4. 이미지 해시 + 캐시 조회	SHA-256 해시 → Redis 조회	< 10ms
5. OCR (캐시 미스 시)	Google Cloud Vision DOCUMENT_TEXT_DETECTION	800~1500ms
6. LLM 구조화	Claude API (sonnet 모델)	1000~3000ms
7. 식약처 DB 매칭	PostgreSQL 풀텍스트 검색	50~200ms
8. 결과 저장·캐시	PostgreSQL INSERT + Redis SET	< 50ms
9. 응답 반환	JSON 직렬화 → HTTPS	100~300ms
합계 (캐시 미스)		약 2.5~6초
합계 (캐시 히트)		< 1초

5. 비용 추정 (학생 팀 기준)

5.1 PoC ~ MVP 단계 (10주, 베타 사용자 50명 가정)

항목	사용량	단가	월 비용 (USD)
Google Cloud Vision API	1,500건/월 (100건 무료 차감)	\$1.5/1k	약 \$0.75
Claude API	5,000회 호출 (입력 1k + 출력 200 토큰)	sonnet 가격	약 \$15~25
NCP 백엔드	1 vCPU, 2GB RAM 인스턴스	학생 크레딧 활용	\$0 (크레딧 사용 시)
NCP DB Manager	PostgreSQL 1GB	학생 크레딧	\$0
NCP Object Storage	10GB 사진 저장	\$0.02/GB	약 \$0.20
도메인 (선택)	.com 1년	\$12/년	약 \$1
합계			약 \$17~27/월

5.2 정식 출시 시 (월 1만 활성 사용자 가정)

항목	월 비용 (USD)
Cloud Vision API	\$50~150
Claude API	\$300~800
인프라 (NCP)	\$200~500
합계	약 \$550~1,450/월 (~ 70~190만원)

💡 **비용 절감 전략:** 1. OCR 결과 캐싱 (동일 영양제 = 동일 결과) → API 호출 50%+ 절감 2. KDRIs 룩업은 PostgreSQL에 저장, LLM 호출 최소화 3. CLOVA OCR이 더 저렴할 경우 폴백 4. 사용량 임계치 도달 시 알림 (NCP/AWS Cloud Watch)

6. 의사결정 매트릭스 — 왜 이 스택인가

10주 학생 팀 + 발표 + 양대 스토어 배포라는 제약 조건에서 각 후보를 평가:

평가 기준 (가중치)	Flutter	FastAPI	PostgreSQL	Cloud Vision	Claude	합계
학습 곡선 (25%)	8/10	9/10	8/10	9/10	9/10	8.6
개발 속도 (20%)	9/10	9/10	8/10	10/10	9/10	9.0
비용 효율 (15%)	10/10	10/10	10/10	8/10	7/10	9.0
확장성 (15%)	8/10	9/10	10/10	9/10	9/10	9.0
커뮤니티·문서 (10%)	9/10	9/10	10/10	10/10	9/10	9.4
유지보수성 (10%)	8/10	9/10	10/10	10/10	9/10	9.2
발표 어필 (5%)	9/10	8/10	7/10	9/10	10/10	8.6
종합	8.7	9.0	8.9	9.2	8.8	8.9/10

💡 모든 영역에서 평균 8.5 이상 — 균형 잡힌 스택임이 정량적으로 검증됨.

7. 학습 자료

팀원이 빠르게 익힐 수 있도록 권장 자료를 정리:

Flutter

- 공식 튜토리얼: <https://docs.flutter.dev/get-started/codelab>
- 한국어: 인프런 “Flutter 만들기” (저자: 코드팩토리)
- `health` 패키지 예제: <https://pub.dev/packages/health>

FastAPI

- 공식 튜토리얼 (한국어 번역 있음): <https://fastapi.tiangolo.com/ko/>
- 책: “FastAPI를 사용한 견고한 파이썬 웹 API 개발” (한빛미디어)

Google Cloud Vision

- 공식 빠른 시작: <https://cloud.google.com/vision/docs/quickstart-client-libraries>
- 한국어 강의: 인프런 “GCP Vision API 활용”

Claude API

- 공식 문서: <https://docs.claude.com/>
- Tool Use 가이드: <https://docs.claude.com/en/docs/build-with-claude/tool-use>

- 프롬프트 엔지니어링: <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

PostgreSQL + TimescaleDB

- TimescaleDB 5분 시작: <https://docs.tigerdata.com/getting-started/latest/>
- PostgreSQL Tutorial: <https://www.postgresqltutorial.com/>

8. 확장·마이그레이션 가능성

향후 사용자·기능 증가 시 어떻게 확장할 수 있는가.

8.1 단기 (Year 1)

- 모바일: Flutter 그대로 유지, 웹 버전(Flutter Web) 추가 가능
- 백엔드: Uvicorn workers 수직 확장
- DB: Read Replica 추가

8.2 중기 (Year 2~3)

- 마이크로서비스 분리 (필요 시): 알고리즘 / OCR / 사용자 / 추천을 별도 서비스로
- 비동기 작업 큐 도입: Celery + RabbitMQ (대량 OCR 배치)
- CDN 도입: 정적 자원 + 이미지
- 카프카 도입: 실시간 헬스 데이터 스트리밍

8.3 장기 — LDB 통합

레몬헬스케어의 LDB 의료 데이터 플랫폼과 연동 시: - HL7 FHIR KR Core 표준 적용 - 마이데이터 채널 통합 (보건복지부 사업) - AES-256 컬럼 암호화, ISMS-P 인증 필수

🔍 법규·표준 상세: [10-compliance-checklist.md](#)

8.4 마이그레이션 가능성

영역	마이그레이션 시 영향
Flutter → React Native	전체 재작성 (높은 비용)
FastAPI → Django	알고리즘 코드는 재사용 가능, 라우터·미들웨어만
PostgreSQL → MySQL	SQL 미세 조정 (대부분 호환)
Cloud Vision → CLOVA	OCR 호출 부분만 교체 (인터페이스 추상화로 쉽게)
Claude → GPT	API 클라이언트만 교체 (프롬프트는 동일)

💡 설계 원칙: 외부 API(OCR·LLM)는 **Adapter 패턴**으로 추상화 → 향후 교체 비용 ↓.

```
# 예시: OCR Adapter 패턴
class OCRAdapter(ABC):
    @abstractmethod
    async def extract_text(self, image_bytes: bytes) -> str: ...

class GoogleVisionOCR(OCRAdapter): ...
class CLOVAOCR(OCRAdapter): ...
class TesseractOCR(OCRAdapter): ...

# 사용처는 변경 없음
ocr: OCRAdapter = GoogleVisionOCR() # ← 한 줄만 바꾸면 교체
text = await ocr.extract_text(image)
```

 변경 이력

버전	날짜	변경 사항	작성자
v1.0	2026-05-03	초안 작성. 5개 핵심 + 부가 스택, 의사 결정 매트릭스 포함	TBD

 관련 문서

- [01. 프로젝트 개요](#)
- [05. GitHub 협업 규칙](#)
- [07. 핵심 알고리즘](#)
- [08. 구현 계획](#)
- [09. 데이터·API 카탈로그](#)
- [10. 컴플라이언스 체크리스트](#)